

ERPChat Complete Reference Dictionary: Business Logic & SQL Rules

Consolidated context blueprint designed to eliminate entity hallucinations, ambiguous filters, and calculation errors within RAG indexing engines.

Section 1: Revenue Interpretation & Sales Calculation Logic

Rule 1.1: Realized Revenue Boundaries & Status Filters

Revenue is strictly defined as **finalized billed sales only**. Operational record staging states do not qualify as real transaction items:

- **Default Omissions:** Always exclude orders where `sales_orders.status IN ('Draft', 'Cancelled')` from standard business metrics.
- **Exception Clause:** Include draft/cancelled entities *only* if the prompt explicitly asks for "all orders" or specifies including unfinalized records. When executing this path, the system must clearly output an explicit notification note highlighting that this is a non-default financial interpretation.

```
-- Standard default realized revenue evaluation query
SELECT SUM(total_amount) AS standard_realized_revenue
FROM sales_orders
WHERE status NOT IN ('Draft', 'Cancelled');
```

Rule 1.2: Tax Modifications & Invoiced View Hierarchies

Total corporate revenue metrics are split cleanly across tax boundaries:

- **Default Total Revenue (Tax-Inclusive):** Computed using final order-level figures: `SUM(sales_orders.total_amount)` . This represents the complete business view.
- **Pre-tax Subtotal Revenue:** Computed using `SUM(sales_orders.subtotal)` . This path is used *only* when explicitly requested by name.
- **Line-Tier Revenue Grouping:** Product metrics are tracked through line configurations using `SUM(order_items.line_total) GROUP BY product_id` . Never mix product-level line lists with top-tier order entries to prevent duplication scaling bugs.

$$line_total = quantity \times unit_price \times (1 - (discount_percent / 100))$$

LLM Guardrail: Never aggregate product baseline `unit_price` metrics out of the master product table for historical metrics. Always fetch price values dynamically from `order_items.unit_price` as it preserves the fixed historical transactional cost profile.

Rule 1.3: Period Boundaries & Financial Reporting Defaults

- **Time Alignment:** For periodic calculations (Yearly, Quarterly, Monthly metrics), match transaction dates against target timelines strictly using `sales_orders.order_date` context constraints.
- **KPI Core Meaning:** Preservation of absolute accounting logic metrics takes precedence over formatting. Summary details must match exact balance states even if display layouts are truncated.

Conservative Default Framework: Whenever any user question context remains ambiguous or poorly specified, the system must favor conservative accounting assumptions: realize revenue from finalized fields only, strip Draft/Cancelled statuses, preserve tax calculations, and append a clear structural assumption notice clarifying the parameters used.

Section 2: Credit Risk, Customer Profiles & INR Conventions

Rule 2.1: Indian Currency Numbering (Lakh Mapping)

Account credit limit comparisons are handled relative to Indian National Rupee (INR) numerical standards. Whenever a threshold query mentions "lakhs", transform the text to absolute numerals before evaluating clauses:

- **1 Lakh** = 100,000
- **5 Lakhs** = 500,000

Rule 2.2: Credit Limit Boundaries vs Outstanding Balances

Keep account balance data parameters distinctly mapped:

- **High-Credit Queries:** Apply conditional parameters explicitly against `customers.credit_limit` using the calculated threshold.
- **Outstanding Debts:** Query `customers.outstanding_balance` parameters directly. Do not substitute or overlap debt balances with overall active lines of credit.
- **Credit Violation Formula:** An active account falls out of clear guidelines when:

`outstanding_balance > credit_limit`

```
-- Query to isolate at-risk customer profiles
SELECT customer_id, company_name, credit_limit, outstanding_balance
FROM customers
WHERE is_active = true AND outstanding_balance > credit_limit;
```

Section 3: Inventory Architecture, SKUs & Facilities

Rule 3.1: SKU Resolution and Facility Enumeration

- **SKU Inquiries:** Filter records directly through text strings matches on `products.sku` and map physical metrics via `product_id` table joins.
- **Warehouse Count Metric:** To compute structural counts cleanly, always apply distinct evaluations: `COUNT(DISTINCT inventory.warehouse_name)`.

Rule 3.2: Available Quantities & High-Velocity Parameters

- **Real Available Stock:** Calculated per-warehouse via $available_stock = stock_on_hand - allocated_stock$.
- **Low Stock Replenishment Flag:** Active when $available_quantity \leq reorder_level$.
- **Fast-Moving / Velocity Ranking:** Sort items according to active sales data volume inside current year timeline boundaries, aggregating through: `SUM(order_items.quantity) AS units_sold`.

Section 4: Structural Guardrails & Double-Entry Verification

Rule 4.1: Cross-Domain Boundary Separation

Absolute Structural Guardrail: Customer parameter modules (credit, details) and Warehouse parameter modules (stock levels, facilities) represent isolated data blocks. Never join `customers` to `inventory` unless cross-domain criteria are explicitly required.

Rule 4.2: The General Ledger Balance Equation

Double-entry bookkeeping demands absolute matching consistency. Grouped across any specific unique entry event, debits must balance with credits precisely:

$$\Sigma(debit) = \Sigma(credit)$$

If `SUM(debit) != SUM(credit)` evaluates to true for a transaction transaction group, a system-level ledger out-of-balance health warning condition is present.